



CMC UNIVERSITY

Aspire to Inspire the Digital World

Machine Learning and Data Mining

Faculty of Information and Communication Technology

Transformer

Transformer is the core architecture behind modern AI, powering models like ChatGPT and Gemini. Introduced in 2017, it revolutionized how AI processes information. The same architecture is used for training on massive datasets and for inference to generate outputs.

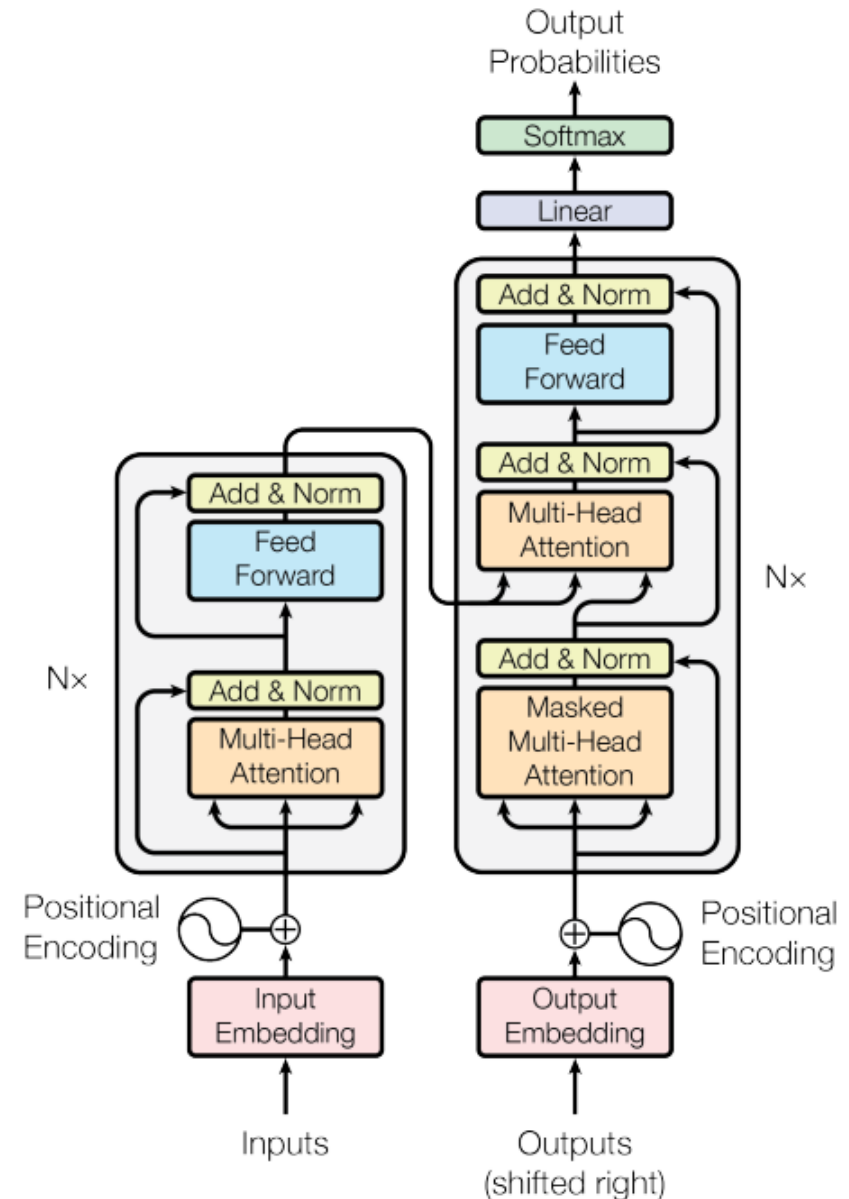
Transformer

Transformer architecture consists of three main parts:

1. Embeddings (with positional encoding) convert tokens into vectors.
2. Transformer blocks use multi-head self-attention to mix information and feedforward networks (MLPs) to refine representations.
3. A final linear layer with softmax outputs probabilities for the next token.

Introduction to Transformer

- How ChatGPT works ?
Transformers Neural Nets...
- Processing inputs in parallel.
- With LSTM, for a large corpus of text, the time increases.
- Transformer is a model that uses self-attention to boost the speed.



Introduction to Transformer

Transformer is a neural network architecture used for various machine learning tasks, especially in natural language processing and computer vision. It focuses on understanding relationships within data to process information more effectively.

- Uses attention mechanisms to capture relationships between inputs
- Processes entire sequences at once instead of step by step
- Improves performance on tasks involving context and dependencies
- Widely used across NLP, vision and other AI applications

Core Concepts Of Transformers

Transformers are powerful models that rely on attention mechanisms and sequence representations to process data efficiently.

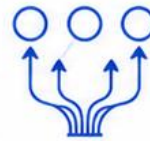
01



Self-Attention

Each position in the sequence attends to all positions in the same sequence to capture contextual relationships.

02



Multi-Head Attention

Runs multiple attention mechanisms in parallel to capture different types of relationships.

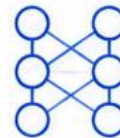
03



Positional Encoding

Adds position information to input embeddings since transformers have no inherent sense of order.

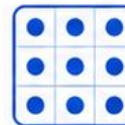
04



Position-wise Feed-Forward Networks

Applies the same feed-forward network independently to each position to add non-linearity.

05



Embeddings

Convert tokens (words, subwords, etc.) into dense vectors that capture semantic meaning.

06



Encoder-Decoder

Encoder processes the input sequence, and decoder generates the output sequence based on the encoder's representation.

Positional Encoding

Unlike RNNs, transformers lack an inherent understanding of word order since they process data in parallel. To solve this problem Positional Encodings are added to token embeddings providing information about the position of each token within a sequence.

Using sine and cosine functions of different frequencies:

- $PE(pos, 2i) = \sin(pos / 10000^{2i / d_{model}})$
- $PE(pos, 2i+1) = \cos(pos / 10000^{2i / d_{model}})$

Position-wise Feed-Forward Networks (FFN)

- The Feed-Forward Networks consist of two linear transformations with a ReLU activation. It is applied independently to each position in the sequence. This transformation helps refine the encoded representation at each position.

$$FFN(x) = \max(0, xW_1 + b_1)W_2 + b_2.$$

Residual Connections and Layer Norm

- **Add & Norm:** Each sub-layer (Attention, FFN) is followed by: $LayerNorm(x + Sublayer(x))$.
- Helps with gradient flow and stabilizes training

The Encoder Block

- Multi-Head Self-Attention.
- Position-wise FFN.
- Both followed by Residual + Layer Norm.

The Decoder Block (Unique Features)

- Masked Multi-Head Attention: Prevents positions from attending to subsequent positions (preserving auto-regressive property).
- Encoder-Decoder Attention: Query from Decoder, Keys/Values from Encoder output.

Transformer illustration in machine translation

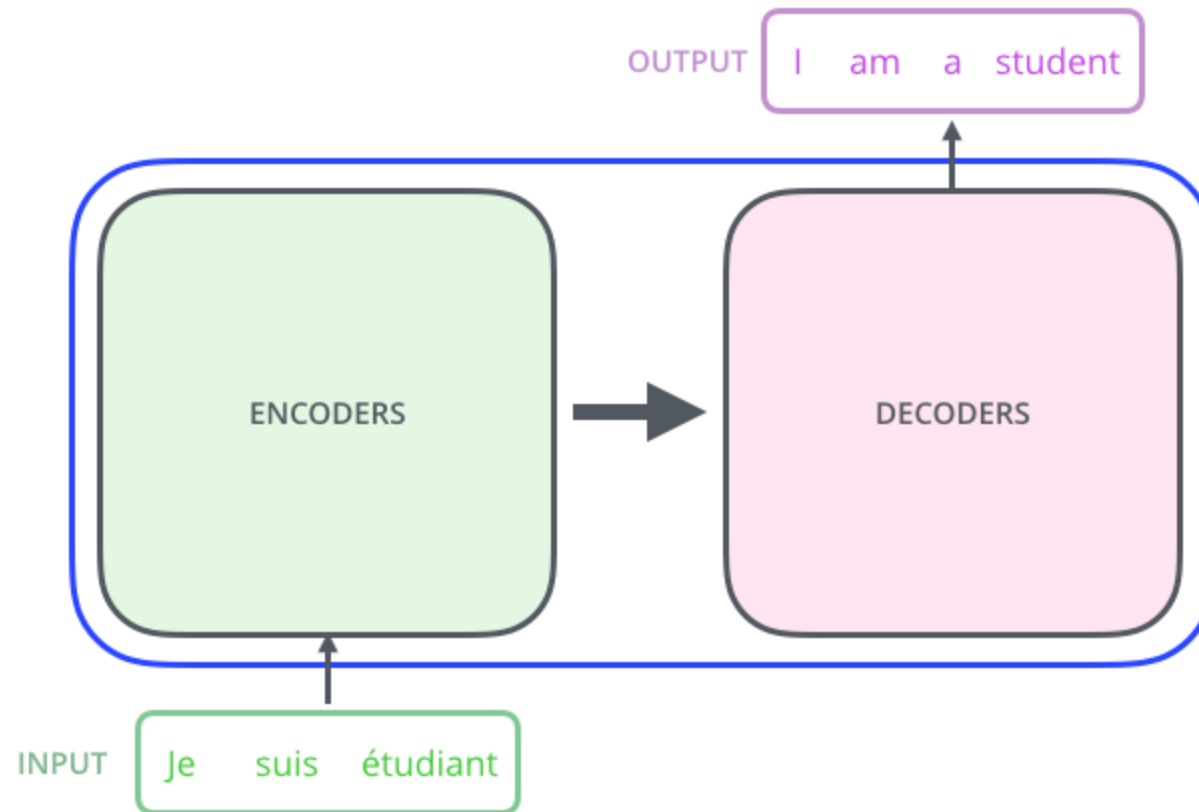
Give a sentence in one language, and output its translation in another.



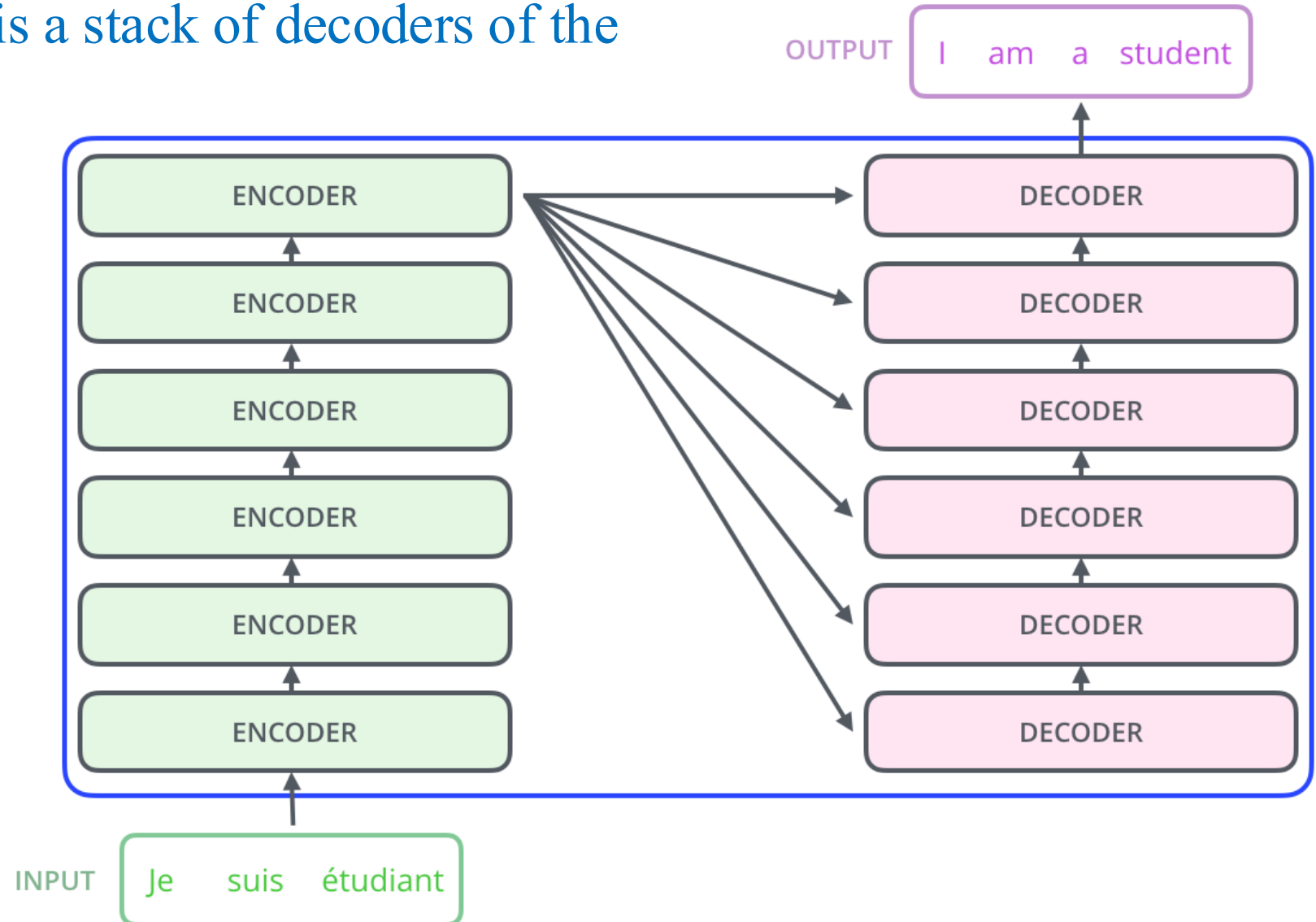
<https://jalammar.github.io/illustrated-transformer/>

Transformer illustration in machine translation

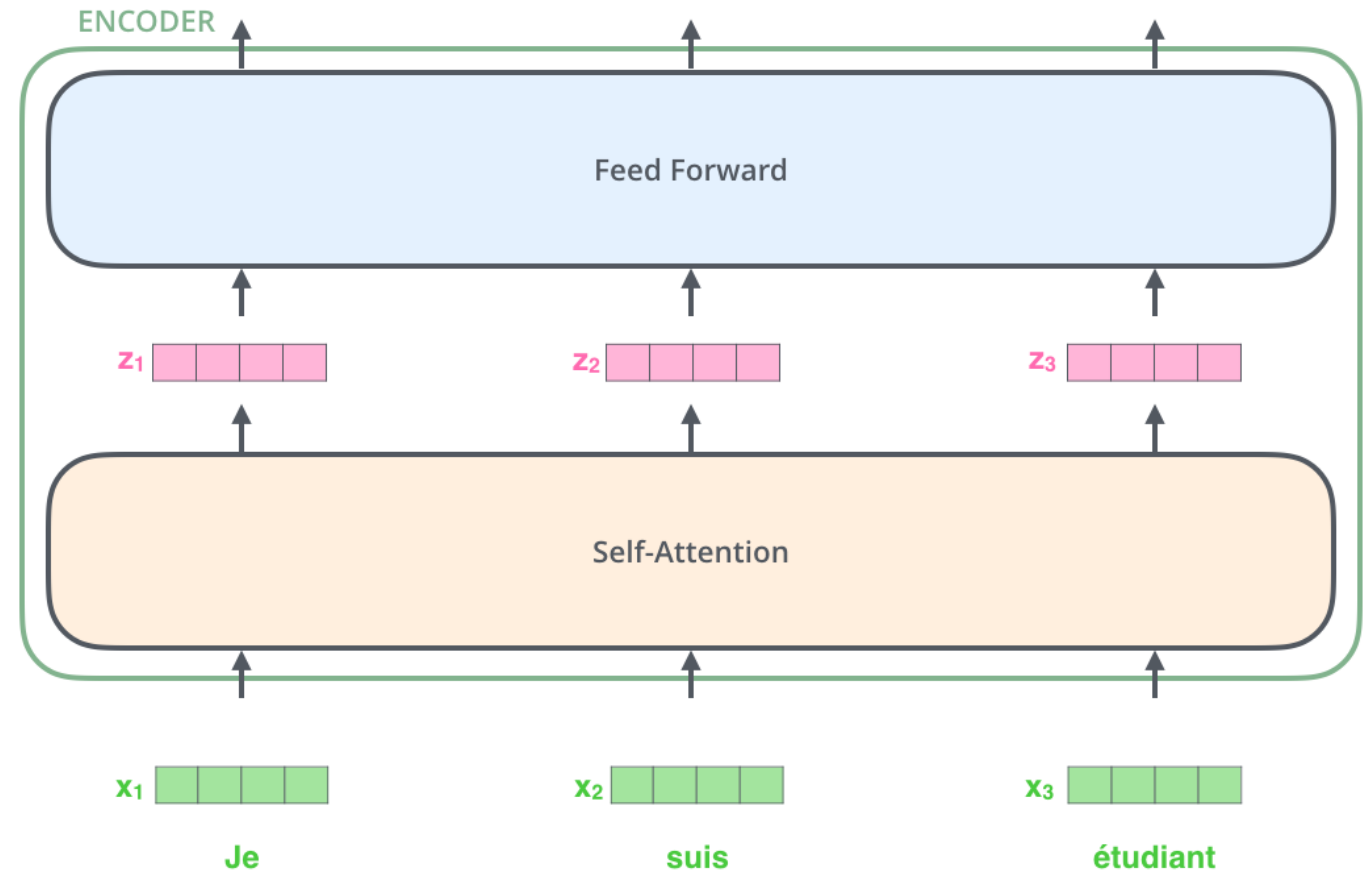
Including: encoding component, a decoding component, and connections between them.



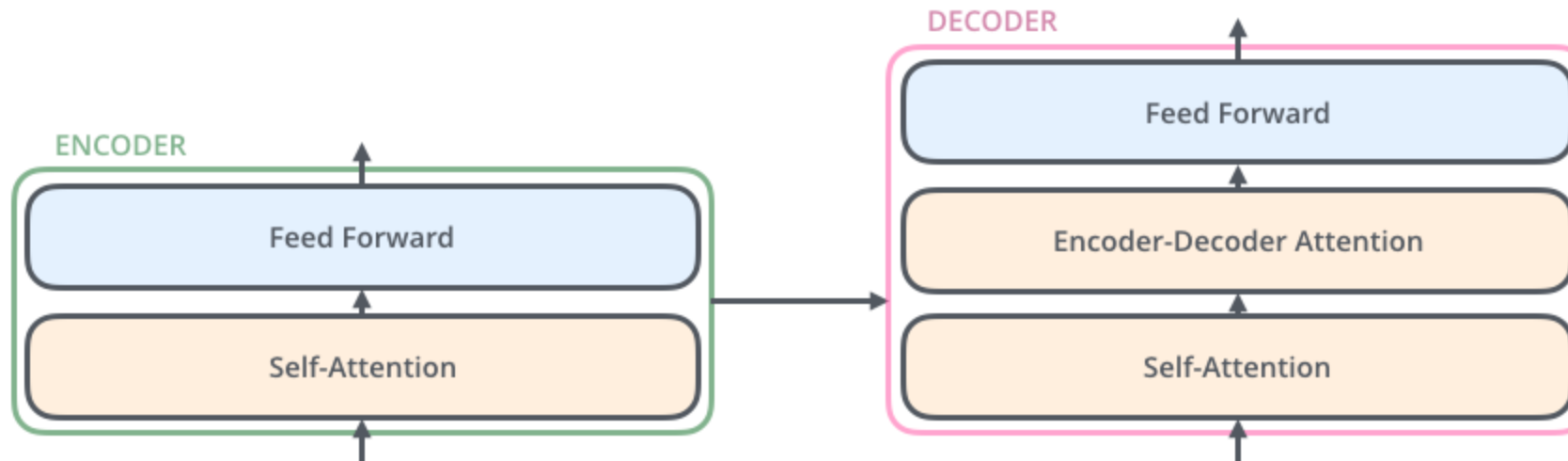
The encoding component is a stack of encoders. The decoding component is a stack of decoders of the same number.



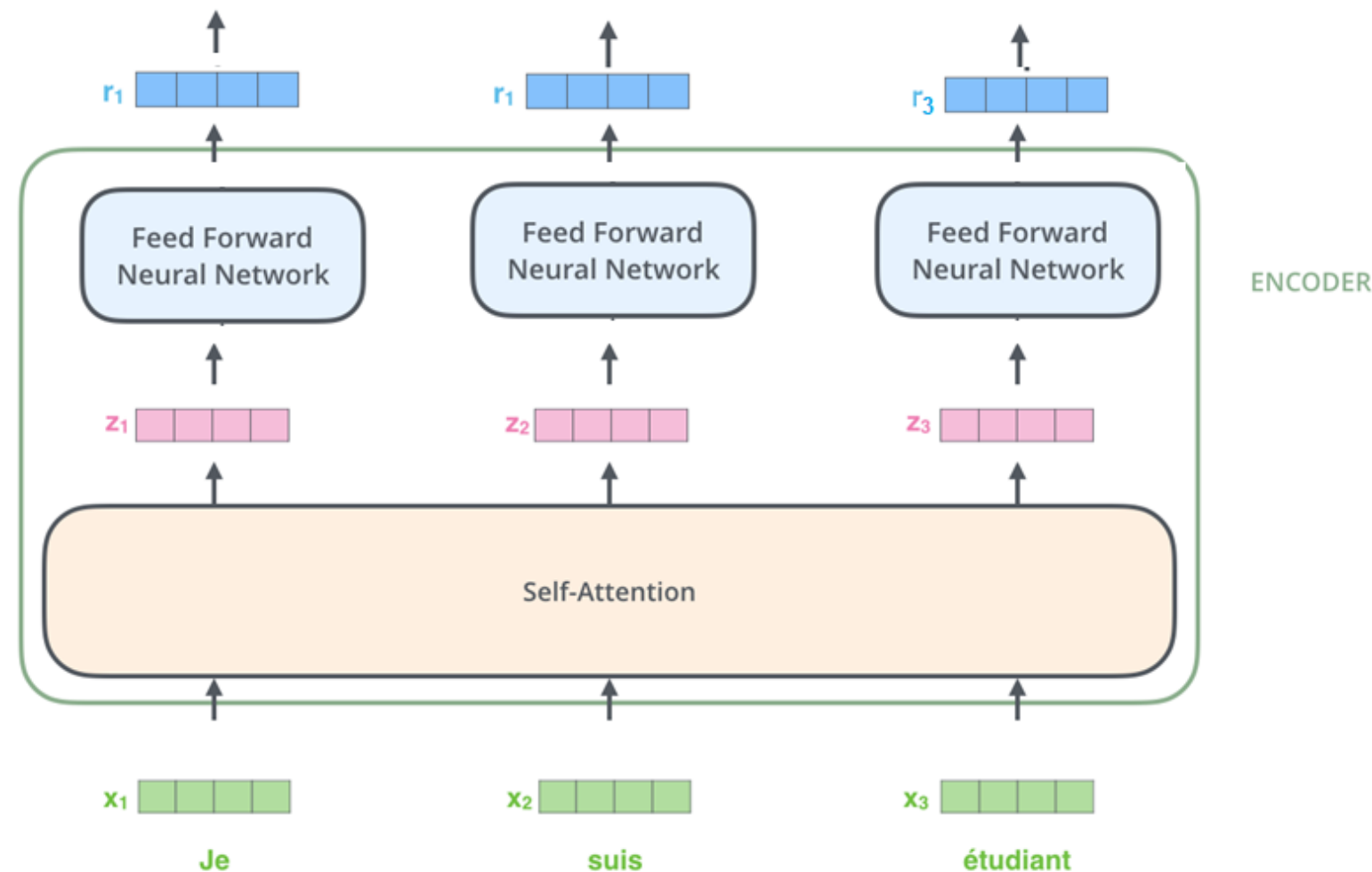
- Each encoder class consists of 2 subclasses.
- The self-attention layer helps the encoder consider other words in the input sentence when encoding a certain word in the sentence.



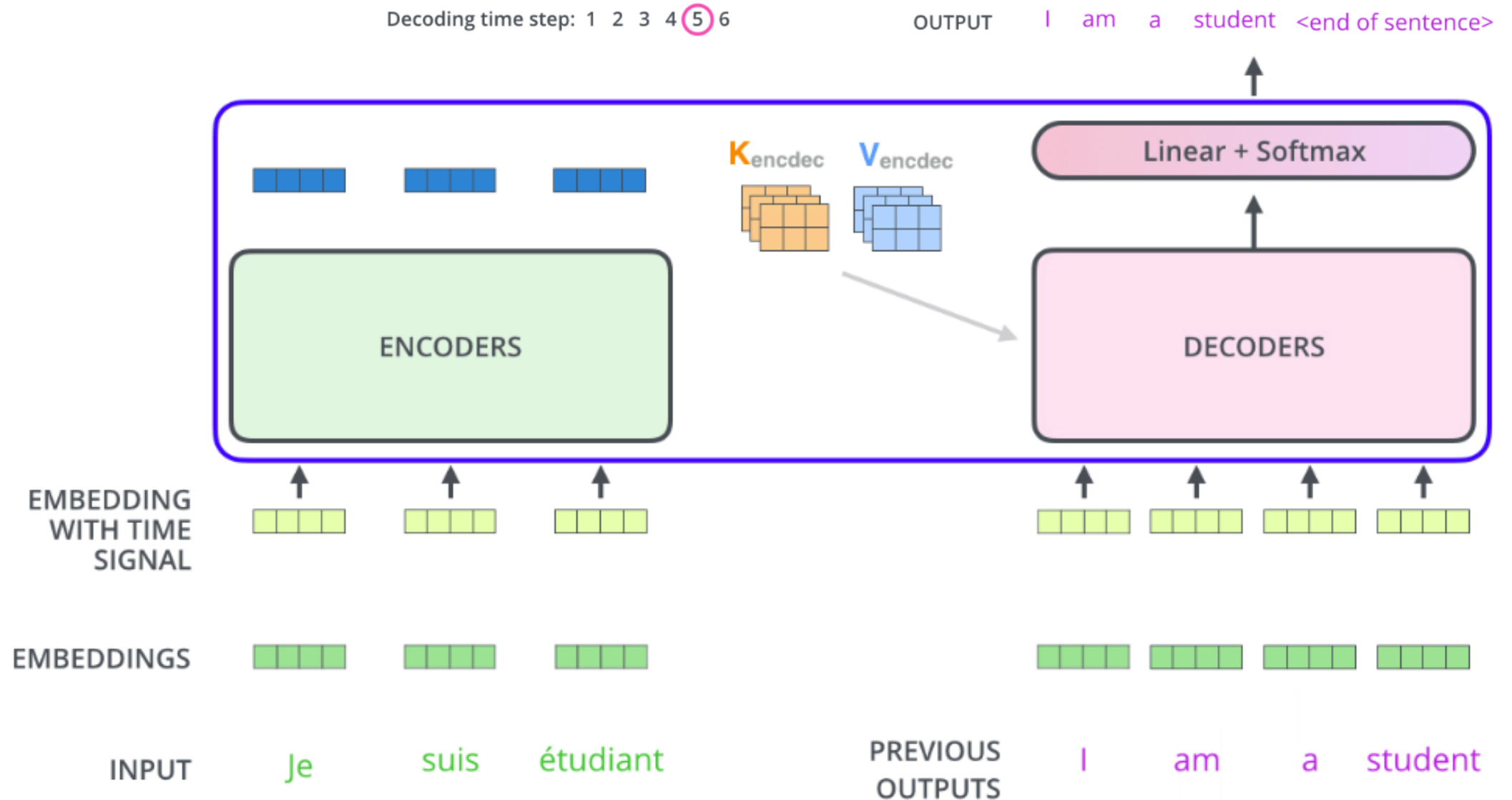
- Self-attention layer helps the encoder look at other words in the input sentence as it encodes a specific word.
- Attention layer that helps the decoder focus on relevant parts of the input sentence.



- After embedding the words in our input sequence, each of them flows through each of the two layers of the encoder.



Each step in the decoding phase outputs an element from the output sequence



TRANSFORMER EXPLAINER

Examples ▾

Artificial Intelligence is transforming the world

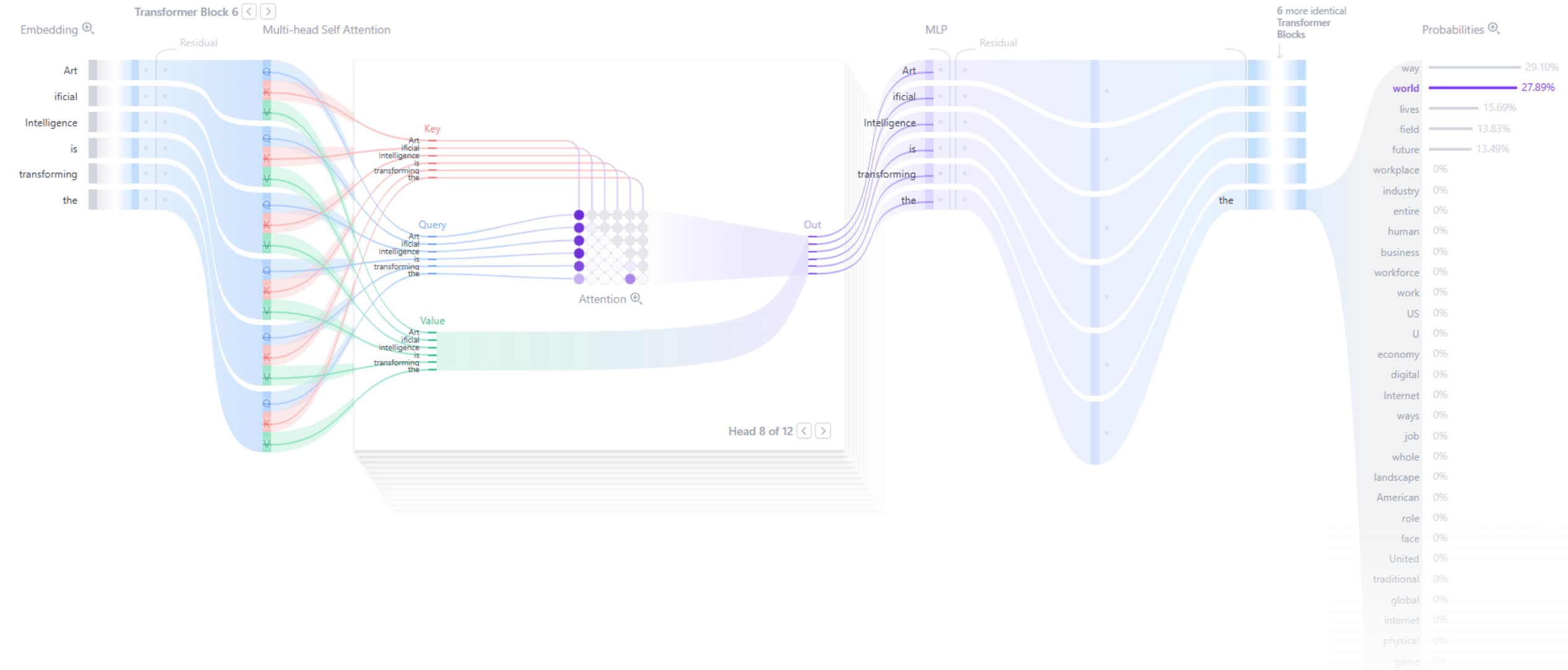
Generate

Temperature 4

Sampling ☒ Top-k ☐ Top-p k=5



Try the examples while GPT-2 model is being downloaded (600MB)



Transformer Advantages

- Parallelization: Unlike RNNs, all tokens are processed simultaneously.
- Long-range dependencies: Constant path length between any two positions.

The Transformer is the backbone for models like BERT (Encoder-only) and GPT (Decoder-only).

Practical Application: Mini-project

- **Task:** Implement a simple Transformer for Machine Translation or Sentiment Analysis.
- **Tools:** Python, PyTorch/Keras.



CMC UNIVERSITY

Aspire to Inspire the Digital World



THANK YOU